

Movie Collection Ecosystem

Backend API & Database Infrastructure Setup Summary

Project Component	Technology Stack / Configurations & Parameters
Backend Solution	.NET Solution with decoupled projects structure managed inside Visual Studio.
Web API Layer	ASP.NET Core Web API (with Controllers), utilizing EF Core compilation tools.
Data Layer	Class Library holding relational definitions, DbSet tracking context and migrations.
Database Engine	PostgreSQL (Cloud Hosted by Supabase Platform).
Connection Mode	Session Pooling Proxy Routing via custom port 6543 to bypass residential ISP blocks.

Chronological Action Items & Technical Resolutions

1. Workplace & Solution Structuring

Initialized a professional decoupled solution topology dividing boundary layers to preserve cross-project integrity.

Commands processed:

```
dotnet new sln -n MovieCatalog
```

```
dotnet new webapi -n MovieCatalog.API --use-controllers
```

```
dotnet new classlib -n MovieCatalog.Data
```

Added cross-referencing so the API can resolve the core domain assemblies: `dotnet add`

`MovieCatalog.API/MovieCatalog.API.csproj reference MovieCatalog.Data/MovieCatalog.Data.csproj`

2. Configuration of Entity Models

Created `Movie.cs` and configured relational annotations using data annotations mapping to a clean, lowercase database schema identifier named `[Table("Movie_Collection")]`. Built custom database mapping configurations inside `DataContext.cs` wrapping around EF Core's infrastructure rules.

3. Assembly Alignment & Version Synchronization

Encountered runtime syntax errors due to duplicate access modifier definitions (`public protected override OnModelCreating`) and runtime compilation warnings regarding target assembly mix-ups. Synchronized solution dependencies to match uniform frameworks (EF Core 10/8 environments). Resolving packages included:

`Microsoft.EntityFrameworkCore`, `Microsoft.EntityFrameworkCore.Design`, and

`Npgsql.EntityFrameworkCore.PostgreSQL`.

4. Database Architecture & Proxy Pool Routing

Configured `appsettings.json` securely pointing to Supabase cloud ecosystem. Faced direct IPv6 network stream interruptions (timeouts running `SELECT 1` routines) caused by firewall and local ISP blocks. Overcame this block by migrating connection layouts away from port 5432 to the explicit **Session Pooler Endpoint** running on port **6543**, enforcing SSL encryption parameter requirements: `SSL Mode=Require;Trust Server Certificate=true;`.

5. Code-First Migration Generation & SQL Deployment

Installed Microsoft.EntityFrameworkCore.Tools to surface compilation shortcuts directly inside Visual Studio Package Manager Console. Successfully compiled and verified the structural database blueprint code-first schema template via:

```
Add-Migration InitialCollectionSetup -Project MovieCatalog.Data -StartupProject MovieCatalog.API
```

To fully step around proxy socket restrictions during raw migration assemblies execution routines, produced clean raw SQL code directly to execution windows using:

```
Script-Migration -Project MovieCatalog.Data -StartupProject MovieCatalog.API
```

Pasting and executing this script inside the Supabase cloud workspace dashboard completed the live schema setup.